1. **Define a base class Crop with attributes like name, area, season, and yieldPerAcre. Use appropriate access specifiers and constructors.**

Ans:-

```
 #include <iostream>
#include <string>
Using namespace std;

Class Crop {
Protected:
   String name;
   Float area;
   String season;
   Float yieldPerAcre;

Public:
   Crop() {
      Name = “”;
      Area = 0.0;
      Season = “”;
      yieldPerAcre = 0.0;
   }
   Crop(string n, float a, string s, float y) {
      Name = n;
      Area = a;
      Season = s;
      yieldPerAcre = y;
   }
   Void display() {
      Cout << “Crop Name: “ << name << endl;
      Cout << “Area: “ << area << “ acres” << endl;
      Cout << “Season: “ << season << endl;
      Cout << “Yield per Acre: “ << yieldPerAcre << endl;
   }
};

Int main() {
   Crop c1(“Rice”, 5.0, “Kharif”, 20.5);
   C1.display();

   Return 0;
```

```
}
```

OUTPUT:
Crop Name: Rice
Area: 5 acres
Season: Kharif
Yield per Acre: 20.5

2. **Derive two classes CerealCrop and vegetableCrop from Crop Using inheritance, adding theirs pacific data members (e.g. fertilizer type, irrigation frequency).**
Ans:-

```
#include <iostream>
#include <string>
Using namespace std;

Class Crop {
Protected:
   String name;
   Float area;
   String season;
   Float yieldPerAcre;

Public:
   Crop(string n, float a, string s, float y) {
      Name = n;
      Area = a;
      Season = s;
      yieldPerAcre = y;
   }

   Void display() {
      Cout << “Crop Name: “ << name << endl;
      Cout << “Area: “ << area << “ acres” << endl;
      Cout << “Season: “ << season << endl;
      Cout << “Yield per Acre: “ << yieldPerAcre << endl;
   }
};

Class CerealCrop : public Crop {
```

```
Private:
   String fertilizerType;

Public:
   CerealCrop(string n, float a, string s, float y, string f)
      : Crop(n, a, s, y) {
      fertilizerType = f;
   }

   Void displayCereal() {
      Display();
      Cout << “Fertilizer Type: “ << fertilizerType << endl;
   }
};

Class VegetableCrop : public Crop {
Private:
   Int irrigationFrequency;

Public:
   VegetableCrop(string n, float a, string s, float y, int i)
      : Crop(n, a, s, y) {
      irrigationFrequency = I;
   }

   Void displayVegetable() {
      Display();
      Cout << “Irrigation Frequency: “ << irrigationFrequency
          << “ times per week” << endl;
   }
};

Int main() {
   CerealCrop c1(“Rice”, 5.0, “Kharif”, 20.5, “Urea”);
   VegetableCrop v1(“Tomato”, 2.0, “Rabi”, 15.0, 3);

   Cout << “Cereal Crop Details:” << endl;
   C1.displayCereal();

   Cout << “\nVegetable Crop Details:” << endl;
```

```
    V1.displayVegetable();

    Return 0;
}
```

Output :
Cereal Crop Details:
Crop Name: Rice
Area: 5 acres
Season: Kharif
Yield per Acre: 20.5
Fertilizer Type: Urea

Vegetable Crop Details:
Crop Name: Tomato
Area: 2 acres
Season: Rabi
Yield per Acre: 15
Irrigation Frequency: 3 times per week

3. **Use constructor overloading and a virtual destructor.**
   Ans:-

```
#include <iostream>
#include <string>
Using namespace std;

Class Crop {
Protected:
    String name;
    Float area;
    String season;
    Float yieldPerAcre;

Public:
    Crop() {
        Name = “”;
        Area = 0.0;
        Season = “”;
        yieldPerAcre = 0.0;
```

```
    }
    Crop(string n, float a, string s, float y) {
        Name = n;
        Area = a;
        Season = s;
        yieldPerAcre = y;
    }
    Virtual ~Crop() {
        Cout << “Crop destructor called.” << endl;
    }

    Void display() {
        Cout << “Crop Name: “ << name << endl;
        Cout << “Area: “ << area << “ acres” << endl;
        Cout << “Season: “ << season << endl;
        Cout << “Yield per Acre: “ << yieldPerAcre << endl;
    }
};

Class CerealCrop : public Crop {
Private:
    String fertilizerType;

Public:
    CerealCrop() : Crop() {
        fertilizerType = “”;
    }
    CerealCrop(string n, float a, string s, float y, string f)
        : Crop(n, a, s, y) {
        fertilizerType = f;
    }

    ~CerealCrop() override {
        Cout << “CerealCrop destructor called.” << endl;
    }

    Void displayCereal() {
        Display();
        Cout << “Fertilizer Type: “ << fertilizerType << endl;
    }
```

```
};

Class VegetableCrop : public Crop {
Private:
   Int irrigationFrequency;

Public:
   VegetableCrop() : Crop() {
      irrigationFrequency = 0;
   }
   VegetableCrop(string n, float a, string s, float y, int i)
      : Crop(n, a, s, y) {
      irrigationFrequency = I;
   }

   ~VegetableCrop() override {
      Cout << "VegetableCrop destructor called." << endl;
   }

   Void displayVegetable() {
      Display();
      Cout << "Irrigation Frequency: "
          << irrigationFrequency << " times per week" << endl;
   }
};

Int main() {
   CerealCrop c1("Rice", 5.0, "Kharif", 20.5, "Urea");
   VegetableCrop v1("Tomato", 2.0, "Rabi", 15.0, 3);

   Cout << "Cereal Crop Details:" << endl;
   C1.displayCereal();

   Cout << "\nVegetable Crop Details:" << endl;
   V1.displayVegetable();

   Return 0;
}
```

Output:

Cereal Crop Details:
Crop Name: Rice
Area: 5 acres
Season: Kharif
Yield per Acre: 20.5
Fertilizer Type: Urea

Vegetable Crop Details:
Crop Name: Tomato
Area: 2 acres
Season: Rabi
Yield per Acre: 15
Irrigation Frequency: 3 times per week
VegetableCrop destructor called.
Crop destructor called.
CerealCrop destructor called.
Crop destructor called.

4. **Apply runtime polymorphism to calculate and display total yield for Crop types.**
   Ans:-

```
#include <iostream>
#include <string>
Using namespace std;

Class Crop {
Protected:
   String name;
   Float area;
   String season;
   Float yieldPerAcre;

Public:
   Crop() {
      Name = “”;
      Area = 0.0;
      Season = “”;
      yieldPerAcre = 0.0;
   }
```

```
    Crop(string n, float a, string s, float y) {
        Name = n;
        Area = a;
        Season = s;
        yieldPerAcre = y;
    }

    Virtual void displayYield() {
        Cout << “Crop Name: “ << name << endl;
        Cout << “Total Yield: “ << area * yieldPerAcre << endl;
    }

    Virtual ~Crop() {}
};

Class CerealCrop : public Crop {
Private:
    String fertilizerType;

Public:
    CerealCrop(string n, float a, string s, float y, string f)
        : Crop(n, a, s, y) {
        fertilizerType = f;
    }

    Void displayYield() override {
        Cout << “Cereal Crop: “ << name << endl;
        Cout << “Fertilizer Type: “ << fertilizerType << endl;
        Cout << “Total Yield: “ << area * yieldPerAcre << endl;
    }
};

Class VegetableCrop : public Crop {
Private:
    Int irrigationFrequency;

Public:
    VegetableCrop(string n, float a, string s, float y, int i)
        : Crop(n, a, s, y) {
        irrigationFrequency = I;
```

```
    }

    Void displayYield() override {
        Cout << “Vegetable Crop: “ << name << endl;
        Cout << “Irrigation Frequency: “
            << irrigationFrequency << “ times per week” << endl;
        Cout << “Total Yield: “ << area * yieldPerAcre << endl;
    }
};

Int main() {
    CerealCrop c1(“Rice”, 5.0, “Kharif”, 20.5, “Urea”);
    VegetableCrop v1(“Tomato”, 2.0, “Rabi”, 15.0, 3);

    Crop* ptr;

    Ptr = &c1;
    Ptr->displayYield();

    Cout << endl;

    Ptr = &v1;
    Ptr->displayYield();

    Return 0;
}
```

Output:-
Cereal Crop: Rice
Fertilizer Type: Urea
Total Yield: 102.5

Vegetable Crop: Tomato
Irrigation Frequency: 3 times per week
Total Yield: 30

5. **Demonstrate operator overloading to compare two crops based on yield .**
   Ans:-

```
#include <iostream>
#include <string>
```

```
Using namespace std;

Class Crop {
Protected:
   String name;
   Float area;
   String season;
   Float yieldPerAcre;

Public:
   Crop() {
      Name = “”;
      Area = 0.0;
      Season = “”;
      yieldPerAcre = 0.0;
   }

   Crop(string n, float a, string s, float y) {
      Name = n;
      Area = a;
      Season = s;
      yieldPerAcre = y;
   }

   Float totalYield() {
      Return area * yieldPerAcre;
   }

   Bool operator>(Crop c) {
      Return totalYield() > c.totalYield();
   }

   Void display() {
      Cout << “Crop Name: “ << name << endl;
      Cout << “Total Yield: “ << totalYield() << endl;
   }

   Virtual ~Crop() {}
};
```

```
Int main() {
   Crop c1(“Rice”, 5.0, “Kharif”, 20.5);
   Crop c2(“Wheat”, 4.0, “Rabi”, 18.0);

   C1.display();
   C2.display();

   If (c1 > c2) {
      Cout << “Rice has a higher total yield.” << endl;
   } else if (c2 > c1) {
      Cout << “Wheat has a higher total yield.” << endl;
   } else {
      Cout << “Both crops have equal total yield.” << endl;
   }

   Return 0;
}
```

Output:
Crop Name: Rice
Total Yield: 102.5
Crop Name: Wheat
Total Yield: 72
Rice has a higher total yield.

6. **Create an abstract class Advisory that has pure virtual function giveAdvice(). Implement this in derived classes to provide farming tips for different crops.**
   Ans:-

```
#include <iostream>
Using namespace std;

Class Advisory {
Public:
   Virtual void giveAdvice() = 0;

   Virtual ~Advisory() {}
};

Class CerealAdvisory : public Advisory {
Public:
```

```
    Void giveAdvice() override {
        Cout << “Farming Tips for Cereal Crops:” << endl;
        Cout << “1. Use suitable fertilizers for healthy growth.” << endl;
        Cout << “2. Ensure proper irrigation and drainage.” << endl;
        Cout << “3. Monitor crops for pests and diseases.” << endl;
    }
};

Class VegetableAdvisory : public Advisory {
Public:
    Void giveAdvice() override {
        Cout << “Farming Tips for Vegetable Crops:” << endl;
        Cout << “1. Use well-drained and fertile soil.” << endl;
        Cout << “2. Maintain regular irrigation.” << endl;
        Cout << “3. Remove weeds and monitor for pests.” << endl;
    }
};

Int main() {
    CerealAdvisory cereal;
    VegetableAdvisory vegetable;

    Advisory* ptr;

    Ptr = &cereal;
    Ptr->giveAdvice();

    Cout << endl;

    Ptr = &vegetable;
    Ptr->giveAdvice();

    Return 0;
}
```

Output:

```
Farming Tips for Cereal Crops:
1. Use suitable fertilizers for healthy growth.
2. Ensure proper irrigation and drainage.
3. Monitor crops for pests and diseases.
```

Farming Tips for Vegetable Crops:
1. Use well-drained and fertile soil.
2. Maintain regular irrigation.
3. Remove weeds and monitor for pests.

7. **Use file handling to save and retrieve Crop data.**

Ans:-

```
#include <iostream>
#include <fstream>
#include <string>
Using namespace std;

Class Crop {
Private:
   String name;
   Float area;
   String season;
   Float yieldPerAcre;

Public:
   Crop() {
      Name = “”;
      Area = 0.0;
      Season = “”;
      yieldPerAcre = 0.0;
   }

   Crop(string n, float a, string s, float y) {
      Name = n;
      Area = a;
      Season = s;
      yieldPerAcre = y;
   }

   Void saveData() {
      Ofstream file(“crop.txt”, ios::app);

      If (file.is_open()) {
         File << name << endl;
```

```
            File << area << endl;
            File << season << endl;
            File << yieldPerAcre << endl;
            File.close();

            Cout << “Crop data saved successfully.” << endl;
        } else {
            Cout << “Error opening file for writing.” << endl;
        }
    }

    Void retrieveData() {
        Ifstream file(“crop.txt”);

        If (file.is_open()) {
            While (getline(file, name)) {
                File >> area;
                File.ignore();
                Getline(file, season);
                File >> yieldPerAcre;
                File.ignore();

                Cout << “\nCrop Name: “ << name << endl;
                Cout << “Area: “ << area << “ acres” << endl;
                Cout << “Season: “ << season << endl;
                Cout << “Yield per Acre: “
                    << yieldPerAcre << endl;
            }

            File.close();
        } else {
            Cout << “Error opening file for reading.” << endl;
        }
    }
};

Int main() {
    Crop c1(“Rice”, 5.0, “Kharif”, 20.5);
    Crop c2(“Wheat”, 4.0, “Rabi”, 18.0);
```

```
    C1.saveData();
    C2.saveData();

    Crop c3;
    Cout << "\nRetrieved Crop Data:" << endl;
    C3.retrieveData();

    Return 0;
}
```

Output:
Error opening file for writing.
Error opening file for writing.

Retrieved Crop Data:
Error opening file for reading.

8. **Include at least one use of static members (e.g. count of crops).**
   Ans:-

```
#include <iostream>
#include <string>
Using namespace std;

Class Crop {
Private:
    String name;
    Float area;
    String season;
    Float yieldPerAcre;

    Static int cropCount;

Public:
    Crop() {
        Name = "";
        Area = 0.0;
        Season = "";
        yieldPerAcre = 0.0;
        cropCount++;
    }
```

```
    Crop(string n, float a, string s, float y) {
        Name = n;
        Area = a;
        Season = s;
        yieldPerAcre = y;
        cropCount++;
    }

    Void display() {
        Cout << “Crop Name: “ << name << endl;
        Cout << “Area: “ << area << “ acres” << endl;
        Cout << “Season: “ << season << endl;
        Cout << “Yield per Acre: “ << yieldPerAcre << endl;
    }

    Static void displayCount() {
        Cout << “Total number of crop objects: “
             << cropCount << endl;
    }
};

// Definition of static data member
Int Crop::cropCount = 0;

Int main() {
    Crop c1(“Rice”, 5.0, “Kharif”, 20.5);
    Crop c2(“Wheat”, 4.0, “Rabi”, 18.0);
    Crop c3(“Tomato”, 2.0, “Winter”, 15.0);

    C1.display();
    Cout << endl;

    C2.display();
    Cout << endl;

    C3.display();
    Cout << endl;

    Crop::displayCount();
```

```
    Return 0;
}
```

Output :
Crop Name: Rice
Area: 5 acres
Season: Kharif
Yield per Acre: 20.5

Crop Name: Wheat
Area: 4 acres
Season: Rabi
Yield per Acre: 18

Crop Name: Tomato
Area: 2 acres
Season: Winter
Yield per Acre: 15

Total number of crop objects: 3

9. **Ensure appropriate input/ Output formatting and proper exception handling (e.g., invalid area or yield input).**

Ans :-

```
#include <iostream>
#include <string>
#include <stdexcept>
#include <limits>
#include <iomanip>
Using namespace std;

Class Crop {
Private:
    String name;
    Float area;
    String season;
    Float yieldPerAcre;

Public:
```

```
Crop() {
   Name = “”;
   Area = 0.0;
   Season = “”;
   yieldPerAcre = 0.0;
}

Void input() {
   Cout << “Enter crop name: “;
   Getline(cin, name);

   Cout << “Enter area in acres: “;
   If (!(cin >> area))
      Throw invalid_argument(“Area must be a number.”);

   If (area <= 0)
      Throw invalid_argument(“Area must be greater than zero.”);

   Cout << “Enter season: “;
   Cin.ignore(numeric_limits<streamsize>::max(), ‘\n’);
   Getline(cin, season);

   Cout << “Enter yield per acre: “;
   If (!(cin >> yieldPerAcre))
      Throw invalid_argument(“Yield must be a number.”);

   If (yieldPerAcre <= 0)
      Throw invalid_argument(“Yield must be greater than zero.”);
}

Void display() {
   Cout << fixed << setprecision(2);
   Cout << “\n========== Crop Details ==========\n”;
   Cout << “Crop Name       : “ << name << endl;
   Cout << “Area            : “ << area << “ acres” << endl;
   Cout << “Season          : “ << season << endl;
   Cout << “Yield per Acre  : “ << yieldPerAcre << endl;
   Cout << “Total Yield     : “ << area * yieldPerAcre << endl;
   Cout << “==================================\n”;
}
```

```
};

Int main() {
   Crop c;

   Try {
      c.input();
      c.display();
   }
   Catch (const invalid_argument& e) {
      Cout << "\nInput Error: " << e.what() << endl;
   }
   Catch (const exception& e) {
      Cout << "\nError: " << e.what() << endl;
   }

   Return 0;
}
```

Output:

```
Enter crop name: maize
Enter area in acres: 890
Enter season: fall
Enter yield per acre: 67

========== Crop Details ==========
Crop Name       : maize
Area            : 890.00 acres
Season          : fall
Yield per Acre  : 67.00
Total Yield     : 59630.00
```